

NexForge Automation — Delivery Guidebook & User Manual

VPD Technologies

June 30, 2026

CONTENTS

NexForge Automation — Complete Delivery Guidebook & User Manual	
1. What this platform is	
2. Getting started	
2.1 Run it locally (Windows)	
2.2 Configuration (.env)	
3. Architecture	
4. Public website — page by page	
5. The two live workflows (what happens on submit)	
5.1 Contact → Lead	
5.2 Careers → Application	
6. Admin / CRM guide (for staff)	
7. Roles & permissions	
8. Email & notifications	
9. REST API reference (/api/v1/)	
10. Database schema (live, 36 tables)	
Useful SQL (read-only)	
11. Security, SEO, performance (built-in)	
12. Deployment	
13. Maintenance & troubleshooting	
14. Not yet built (future scope)	

NexForge Automation — Complete Delivery Guidebook & User Manual

Everything needed to understand, run, operate, extend, and deploy the NexForge Automation platform. Written for the project team, administrators (HR / Sales / Content), and developers.

1. What this platform is

NexForge Automation is an **enterprise marketing website + lightweight CRM and recruitment system** for an industrial-automation company. One Django application serves three surfaces from one PostgreSQL database:

Surface	URL	Audience
Public website	/	anonymous visitors
REST API	/api/v1/	the site's JavaScript + integrations
Admin / CRM	/admin/	staff (Administrator, Content, Sales, HR, Support)

Stack: **Python / Django 6, Django REST Framework, PostgreSQL (Supabase), Bootstrap 5, vanilla JavaScript**, WhiteNoise, Gunicorn. No SPA framework.

2. Getting started

2.1 Run it locally (Windows)

```
cd project
.venv\Scripts\python.exe manage.py migrate          # set up DB
.venv\Scripts\python.exe manage.py seed             # demo content (optional)
.venv\Scripts\python.exe manage.py seed_careers     # demo job openings (optional)
.venv\Scripts\python.exe manage.py createsuperuser # your admin login
.venv\Scripts\python.exe manage.py runserver
```

Or just double-click `run.bat`. Then open

http://127.0.0.1:8000/SMARTPANTS_PRESERVED_201SMARTPANTS_PRESERVED_202.SMARTPANTS_PRESERVED_203

2.2 Configuration (.env)

Copy `.env.example` to `.env` and fill in. Key variables:

Variable	Meaning
<code>SECRET_KEY</code>	Django crypto key (long random string)
<code>DEBUG</code>	<code>True</code> <code>dev</code> / <code>False</code> <code>prod</code>
<code>DATABASE_URL</code>	PostgreSQL connection string
<code>EMAIL_HOST*</code>	SMTP for sending mail (prod)
<code>SALES_INBOX</code> / <code>HR_INBOX</code>	where enquiry / application notifications go
<code>GOOGLE_MAPS_API_KEY</code>	enables the contact-page map
<code>GOOGLE_ANALYTICS_ID</code>	enables visitor analytics (e.g. <code>G-XXXX</code>)
<code>HERO_VIDEO_URL</code>	optional hero background video
<code>CORS_ALLOWED_ORIGINS</code>	prod allow-list
<code>CSP_REPORT_ONLY</code>	<code>False</code> to enforce Content-Security-Policy in prod

Never commit `.env`. It is git-ignored.

3. Architecture

```
Browser → (prod: Cloudflare → Nginx) → Gunicorn → Django middleware chain
      → URL router → View → Models (ORM) → PostgreSQL (Supabase)
      → HTML page OR JSON → back to Browser
```

Apps (one per business domain):

<u>App</u>	<u>Responsibility</u>
core	shared lookups, all template pages, sitemaps, admin dashboard, seed
projects	portfolio + gallery/videos/deliverables/milestones
services	service catalogue
blog	posts + categories
contact	enquiry (CRM) + email
content	FAQ, gallery, awards, downloads, testimonials
careers	job openings + applications (HR)
accounts	role groups + token login

4. Public website — page by page

Navigation (top bar): [Home](#) · [Projects](#) · [Services](#) · [About](#) · [Gallery](#) · [FAQ](#) · [Blog](#) · [Downloads](#) · [Get a Quote](#). Footer adds [Careers](#), [Privacy](#), [Terms](#), [resources](#).

Page	URL	What it does
Home	/	Animated hero (AI-network + optional video) , business stats , services , industries , featured projects , technology marquee , testimonials
Projects	/projects/	Filter (status / industry / technology) + search + grid/list , cards load from the API
Project detail	/projects/<slug>/	Full case study: value, status, duration, client, overview, challenges, solution, technologies, timeline, related projects
Services	/services/ + /services/<slug>/	Catalogue + detail (benefits, deliverables, technologies)
Blog	/blog/ + /blog/<slug>/	Articles, categories
FAQ	/faq/	Accordion
Gallery / Awards / Downloads	/gallery/ - /awards/ /downloads/	Media + files
Careers	/careers/ + /careers/<slug>/	Open roles + online application with resume upload
Contact	/contact/	Enquiry form (+ map when key set)
Privacy / Terms	/privacy/ - /terms/	Legal
Admin	/admin/	Staff CRM/CMS
API docs	/api/docs/	Swagger UI

5. The two live workflows (what happens on submit)

5.1 Contact → Lead

Visitor fills /contact/ → POST /api/v1/enquiries/ (throttled 5/min, validated)
 → Enquiry saved (status = New) ← source of truth
 → Email to SALES INBOX (sales notified)
 → Auto-reply email to the visitor
 → 201 → on-screen "thanks"
Staff ▶ Admin ▶ Enquiries ▶ assign + move status (New → In progress → Converted → Closed)

5.2 Careers → Application

Candidate opens /careers/<job>/ → fills form + uploads resume
 → POST /api/v1/applications/ (throttled, resume type+size validated)
 → JobApplication saved (status = New)
 → Email to HR INBOX + auto-reply to candidate
 → 201 → "submitted"
HR ▶ Admin ▶ Job applications ▶ move status (New → Shortlisted → Interview → Hired/Rejected)

Both flows are anonymous (no login to apply or enquire). The DB record is the source of truth, so a mail failure never loses a lead or application.

6. Admin / CRM guide (for staff)

Log in at </admin/>. The dashboard shows live counts ([projects](#), [enquiries](#), [blog](#), etc.) and recent enquiries.

Common tasks

- [Add a project: Projects → Add. Fill fields; add gallery images, videos, deliverables and timeline inline; save → live instantly.](#)
 - [Work a lead: Contact → Enquiries → open → set status / assign.](#)
 - [Review an applicant: Careers → Job applications → open → download resume → set status.](#)
 - [Publish a job: Careers → Job openings → Add \(tick “is open”\).](#)
 - [Edit content: Services, Blog, FAQ, Gallery, Awards, Downloads, Testimonials.](#)
-

7. Roles & permissions

Roles are Django Groups (Admin → Authentication → Groups). Assign each staff user to a group (Users → Add → tick “Staff status” → pick group).

<u>Role</u>	<u>Can manage</u>
<u>Administrator</u>	<u>everything</u>
<u>Content Manager</u>	<u>projects, services, blog, content</u>
<u>Sales Manager</u>	<u>enquiries (leads)</u>
<u>Technical Support</u>	<u>enquiries</u>
<u>HR Manager</u>	<u>job openings + applications</u>

A superuser bypasses groups and sees all. There is no public signup — visitors never have accounts.

8. Email & notifications

- Library: Django `send_mail` (synchronous). Dev backend prints to console; prod uses SMTP from `.env`.
 - Triggers: new enquiry (→ sales + auto-reply to sender) and new application (→ HR + auto-reply to candidate). All fail-safe (`fail silently`).
 - No queue/Celery, no SMS/push. Adding a background queue is a future option.
-

9. REST API reference (</api/v1/>)

Read-only public viewsets (GET): [projects](#), [services](#), [blog](#), [faqs](#), [gallery](#), [awards](#), [downloads](#), [testimonials](#), [jobs](#).

Write + staff-gated:

- [POST /enquiries/](#) — public create; staff list/update.
- [POST /applications/](#) — public create (multipart, resume); staff list/update.
- [POST /auth/login/](#) — token login (staff), throttled.

Filtering: [?status=](#), [?industry_slug=](#), [?category=](#), [?search=](#), etc. Pagination: 9 per page. Full interactive docs at </api/docs/>, schema at </api/schema/>.

10. Database schema (live, 36 tables)

The database is PostgreSQL on Supabase. Tables group into framework tables (`auth *`, `django *`, `authtoken token`) and application tables below. File fields (`resume`, `image`, `file`, `thumbnail`) store the path; the actual files live in `MEDIA`.

Application (domain) tables

Table	Key columns	Links
<code>core_industry</code>	<code>name, slug, icon</code>	=
<code>core_technology</code>	<code>name, category, icon</code>	=
<code>core_client</code>	<code>name, slug, logo, website</code>	→ <code>industry</code>
<code>projects_project</code>	<code>title, slug, status, value, duration, team_size, overview, challenges, solution, is_featured</code>	→ <code>industry</code> , → <code>client</code>
<code>projects_project_technologies</code>	(join)	<code>project</code> ↔ <code>technology</code>
<code>projects_projectgallery</code>	<code>image, caption, order</code>	→ <code>project</code>
<code>projects_projectvideo</code>	<code>video_url, title</code>	→ <code>project</code>
<code>projects_projectdeliverable</code>	<code>title</code>	→ <code>project</code>
<code>projects_projectmilestone</code>	<code>title, description, date, order</code>	→ <code>project</code>
<code>projects_beforeaftergallery</code>	<code>before_image, after_image, caption</code>	→ <code>project</code>
<code>services_service</code>	<code>title, slug, short/detailed description, is_active</code>	=
<code>services_servicebenefit</code>	<code>text</code>	→ <code>service</code>
<code>services_servicedeliverable</code>	<code>title</code>	→ <code>service</code>
<code>services_service_technologies / industries</code>	(joins)	<code>service</code> ↔ <code>tech / industry</code>
<code>blog_blogcategory</code>	<code>name, slug</code>	=
<code>blog_blogpost</code>	<code>title, slug, summary, body, published_at</code>	→ <code>category</code> , → <code>author (user)</code>
<code>content_faq</code>	<code>question, answer, category, order, is_active</code>	=
<code>content_galleryitem</code>	<code>title, category, image, order</code>	=
<code>content_award</code>	<code>title, year, description, image</code>	=
<code>content_download</code>	<code>title, file, type</code>	=
<code>content_testimonial</code>	<code>author_name, designation, quote, is_active</code>	→ <code>client</code>
<code>contact_enquiry</code>	<code>name, email, phone, message, status</code>	→ <code>assigned to (user)</code>
<code>careers_jobopening</code>	<code>title, slug, department, location, employment_type, experience, description</code>	=
<code>careers_jobapplication</code>	<code>name, email, phone, resume, cover_letter, status</code>	→ <code>opening</code> , → <code>reviewed by (user)</code>

Framework tables: `auth_user`, `auth_group`, `auth_permission`, `auth_group_permissions`, `auth_user_groups`, `auth_user_user_permissions`, `authtoken token`, `django admin log`, `django content type`, `django migrations`, `django session`.

The full column-level dump (types, PKs, FKs) is in `docs/DATABASE_SCHEMA.md`.

USEFUL SQL (READ-ONLY)

```
-- New leads this month
SELECT name, email, created_at FROM contact_enquiry
WHERE created_at >= date_trunc('month', now()) ORDER BY created_at DESC;

-- Applications per job
SELECT o.title, count(a.id) FROM careers_jobopening o
LEFT JOIN careers_jobapplication a ON a.opening_id = o.id GROUP BY o.title;

-- Featured projects
SELECT title, status FROM projects_project WHERE is_featured = true;
```

11. Security, SEO, performance (built-in)

- Security: CSP (nonce-based), HSTS, secure cookies, X-Frame-Options, rate-limiting, server-side file validation, per-user lead access, secrets in env, custom error pages. `DEBUG=False` in prod.
 - SEO: `sitemap.xml`, `robots.txt`, per-page titles/descriptions, slug URLs.
 - Performance: image lazy-load, pagination, `select related` / `prefetch related` (no N+1), WhiteNoise compressed static, reduced-motion support.
-

12. Deployment

```
pip install -r requirements.txt
python manage.py migrate
python manage.py collectstatic --noinput
gunicorn config.wsgi:application --bind 127.0.0.1:8000 # behind systemd
```

- Set `DJANGO_SETTINGS_MODULE=config.settings.prod` and all env vars.
 - Nginx reverse-proxies to Gunicorn; serves `/static/` and `/media/`.
 - SSL via Certbot. Set `CSP_REPORT_ONLY=False` once verified.
 - Launch checklist: rotate DB password, real SMTP, maps + analytics keys, real images, `createsuperuser`, run `seed / seed careers` if desired.
-

13. Maintenance & troubleshooting

<u>Symptom</u>	<u>Fix</u>
<u>UI looks old after deploy</u>	<u>Bump the <code>?v=</code> asset version / hard-refresh; prod uses hashed static</u>
<u>Page won't load (500)</u>	<u>Check server logs; in dev the traceback shows on screen</u>
<u>Emails not arriving</u>	<u>Dev prints to console; in prod check <code>EMAIL_HOST*</code>; leads are still saved</u>
<u>Tests can't create DB on Supabase</u>	<u>Run with a local engine: <code>DATABASE_URL=sqlite:///memory: ... test</code></u>
<u>"port already in use"</u>	<u>A server is already running; open the URL or use another port</u>
<u>Run tests: <code>python manage.py test</code> (62 tests).</u>	

14. Not yet built (future scope)

Customer login / portal, support/ticketing, async notification queue (Celery + Redis), JWT auth, richer sales pipeline, analytics charts. Careers and customer auto-reply emails are now built. The customer-portal decision is pending business approval and does not block anything above.

End of manual. Keep in sync with the codebase.